

React.js

React.js

ReactJS is a **component-based** JavaScript library used to build dynamic and interactive user interfaces

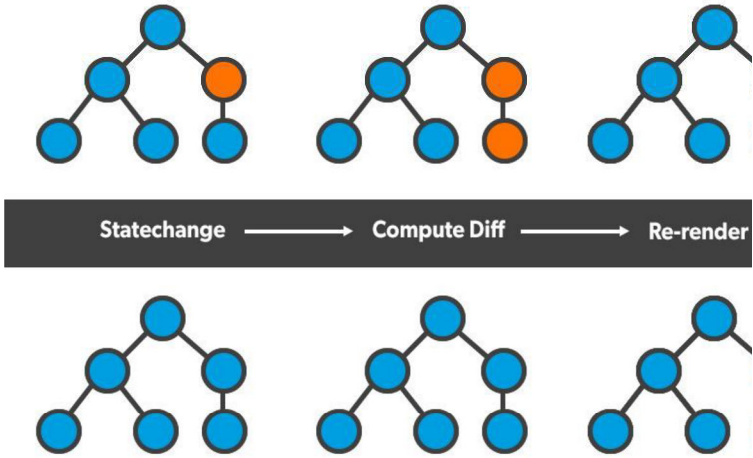
Developed by Facebook Software Engineer Jordan Walke.

It simplifies the creation of **single-page applications (SPAs)** with a focus on performance and maintainability

Allows a **single language** used on the server, client, and mobile sides

React-built products are easy to scale, workflow patterns allow for easy teamwo
UI code is readable and maintainable, and more.

VIRTUAL DOM



DOM

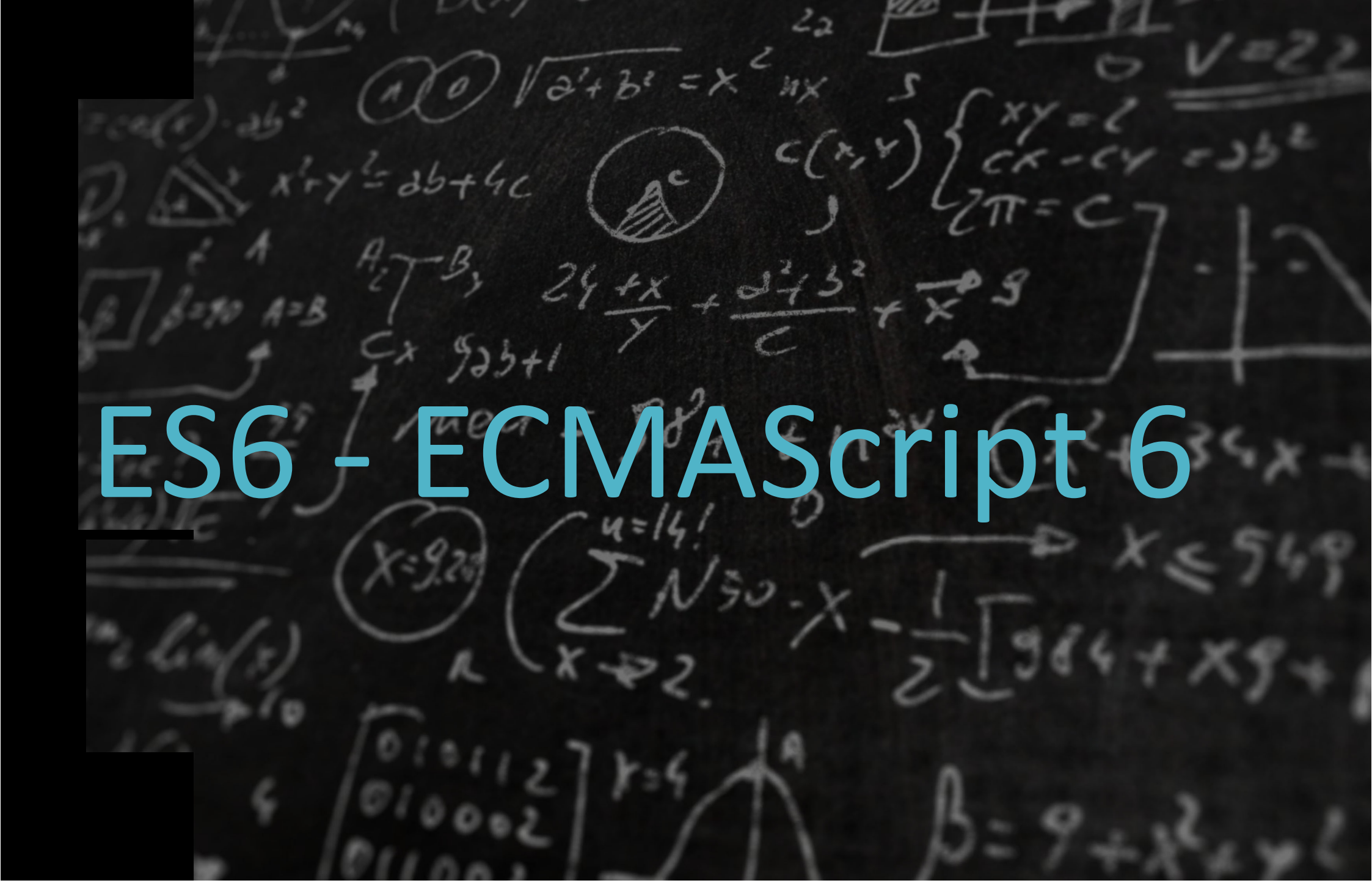
Any changes to the content in the DOM trigger a process called '**reconciliation**' which the browser updates the visible content on the screen

Virtual DOM

React creates a virtual DOM in memory which is the lightweight copy of the real DOM

When there's a change in a React component's state or props,

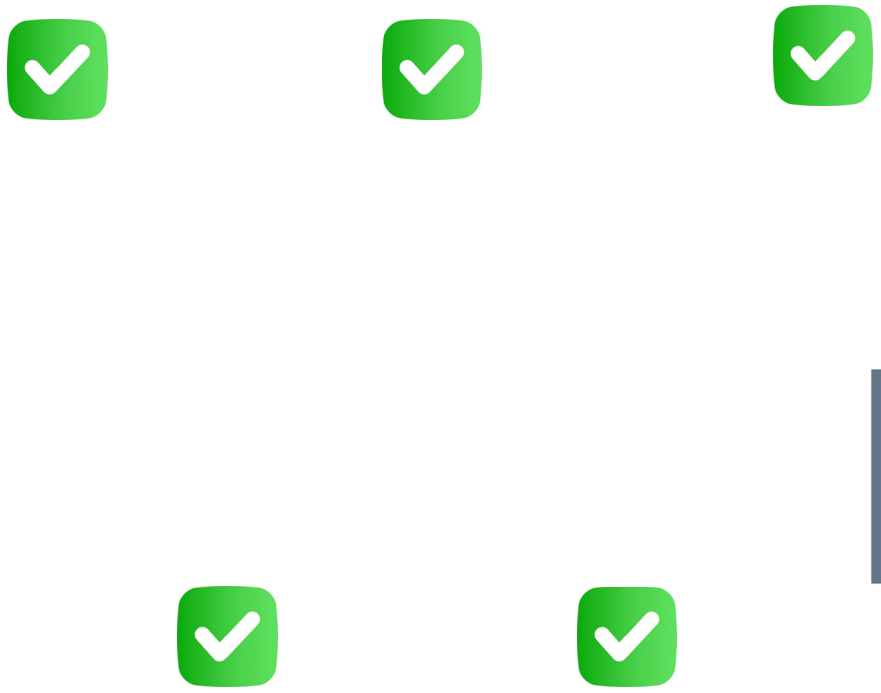
- *React first updates the Virtual DOM to reflect the new state of the component.*
- *identify the differences between the old Virtual DOM and the new Virtual DOM*
- *applies the necessary changes to the real DOM, resulting in an update to user interface*



ES6 - ECMAScript 6

ES6 - ECMAScript 6

ES6 is the 6th version of ECMAScript with some of the new features



Array method – map()

array.map() method used to **create a new array** with the results of calling a fun
for every array element

```
const cel = [10, 20, 30, 40, 50, 60];
```

```
const far = cel.map(x => (x * 9 / 5) + 32);
```

```
[50,68,86,104,122,14
```

```
Cold,Cold,Cold,Cold,
```

```
const isHot = cel.map(x => x > 40 ? "Hot" : "Cold")
```

```
const isHot = cel.map((x, index) => x > 40 ? i
```

```
1-Cold ,2-Cold ,3-Cold ,4-Cold ,5-Hot
```

Array method – filter()

array.filter() used to **create a new array** containing only the elements from the original array that pass a specified condition

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];


// Filter for even numbers
const evenNumbers = numbers.filter(number => number % 2 === 0);
```

```
[2, 4, 6, 8, 10]
```

```
// Filter for numbers greater than 5
const greaterThanFive = numbers.filter(number => number > 5);
```

```
[6, 7, 8, 9, 10]
```

Destructuring

Destructuring allows you to extract values from objects or arrays into **distinct variables**.

```
const vehicles = ['mustang', 'f-150', 'expedition'];
```

```
// old way
const car = vehicles[0];
const truck = vehicles[1];
const suv = vehicles[2];
```

```
// destructuring
const [car, truck, suv] = vehicles;
```

```
// car and suv
const [car,, suv] = vehicles
```

function returns an array

```
function dateInfo(dat) {
  const d = dat.getDate();
  const m = dat.getMonth() + 1;
  const y = dat.getFullYear();

  return [d, m, y];
}

const [date, month, year] = dateInfo(ne
```

Destructuring Objects

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50
};

// Destructuring
let {firstName, lastName, age} = person;

let {lastName, age, firstName} = person;

let {firstName} = person;

// set default value
let {firstName = "Unknown", lastName, age, country= "Unknown"} = person;
```

John Doe 50 Unknown

Spread operator - …

spread operator (...) allows us to quickly copy all or part of an existing array or object into another array or object

```
const num1 = [1, 2, 3];
const num2 = [4, 5, 6];
const numAll = [...num1, ...num2];

console.log(numAll)
```

1,2,3,4,5,6

```
function sum(x, y, z) {
  return x + y + z;
}
// numbers is array
const numbers = [1, 2, 3];

console.log(sum(...numbers)); // 6
```



```
const num = [1, 2, 3, 4, 5, 6

const [one, two, ...rest] = nu

console.log(one)

console.log(two)

console.log(rest)
```

1
2
3,4,5,6

spread operator with objects

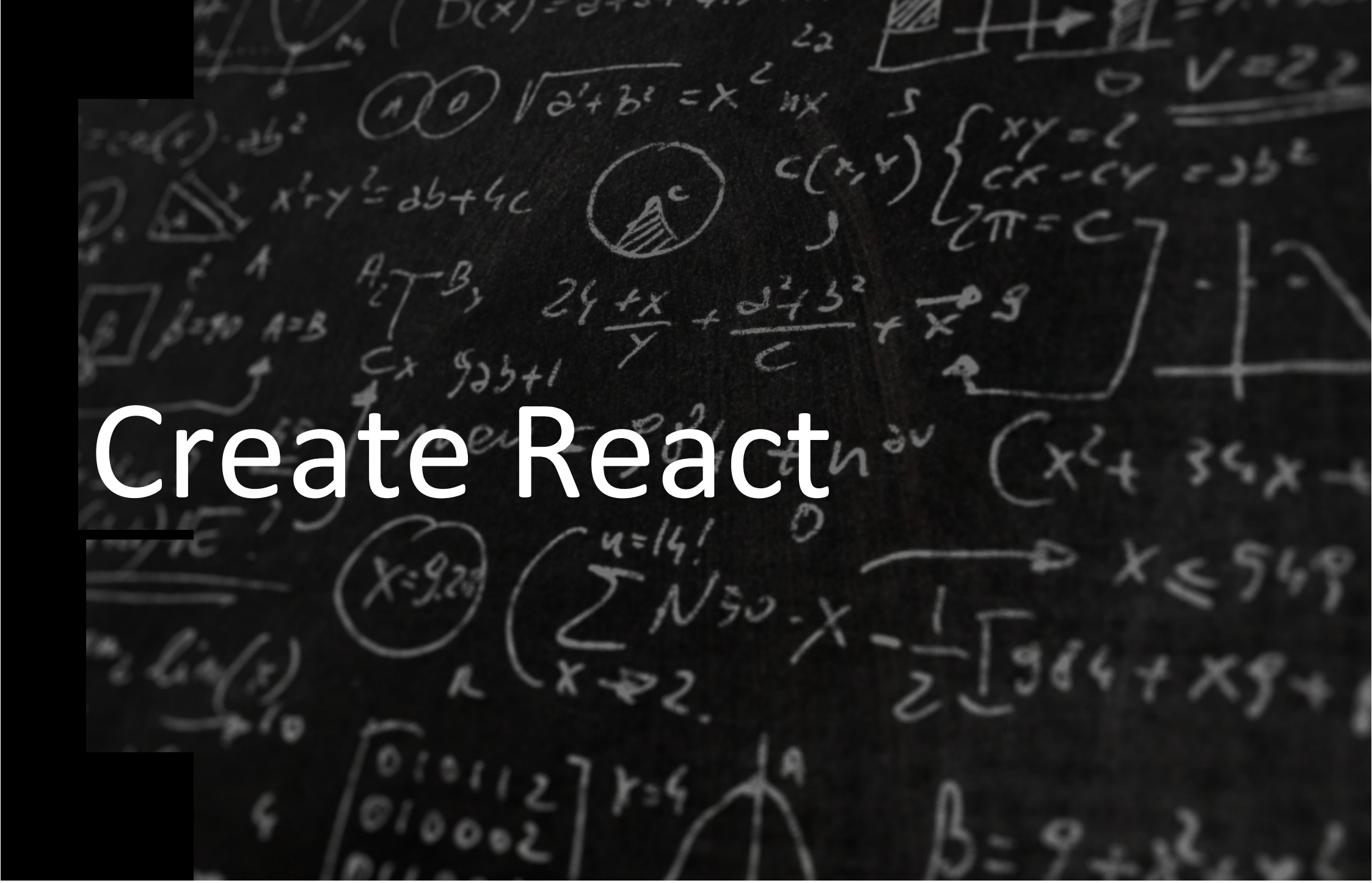
```
const car = {
  brand: 'Ford',
  model: 'Mustang',
  color: 'red'
}

const car_more = {
  type: 'car',
  year: 2021,
  color: 'yellow'
}

const mycar = {...car, ...car_more}
```



```
mycar = {
  brand: "Ford",
  color: "yellow",
  model: "Mustang",
  type: "car",
  year: 2021
}
```



Create React

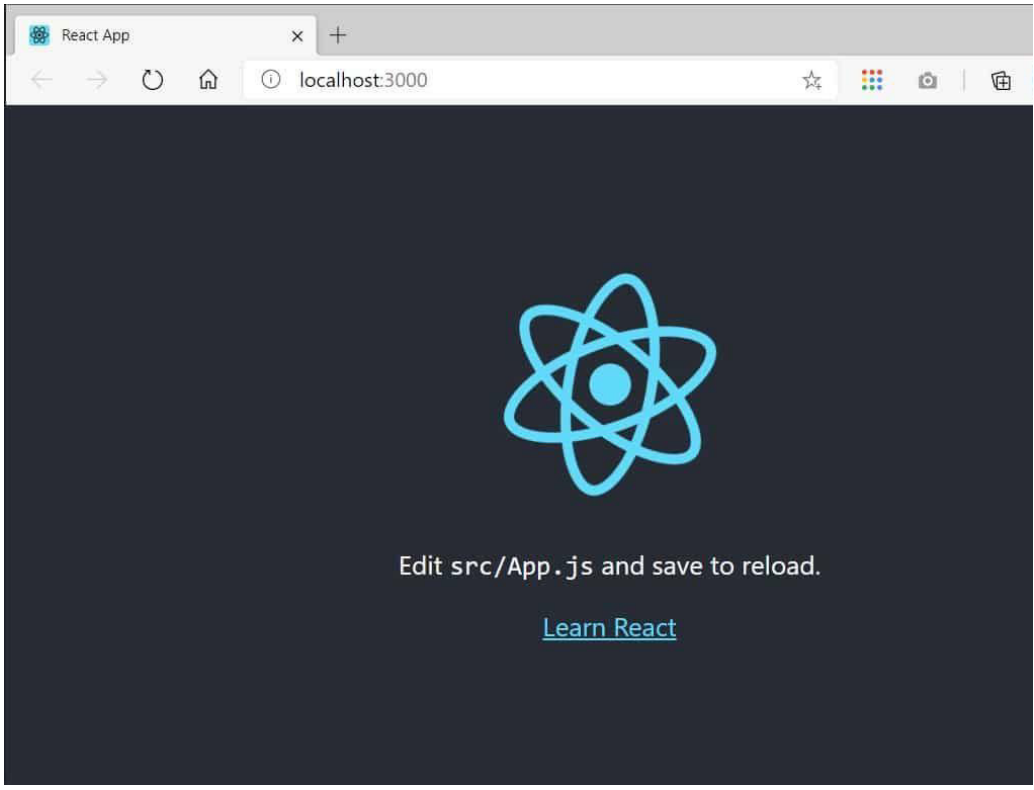
Create React App - Create React App

npm install -g create-react-app

npx create-react-app myreact

npm start

http://localhost:3000/



Create React App - Vite

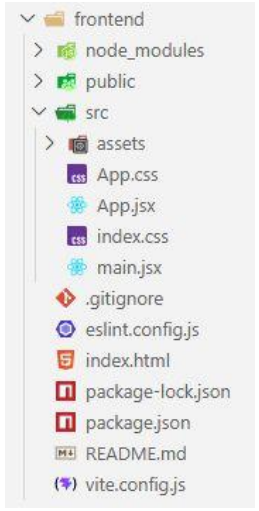
Install Vite

npm install -g create-vite

Create application

npm create vite@latest

Project Name : vite-project
Framework : React
Variant : JavaScript
Linter : ESLint

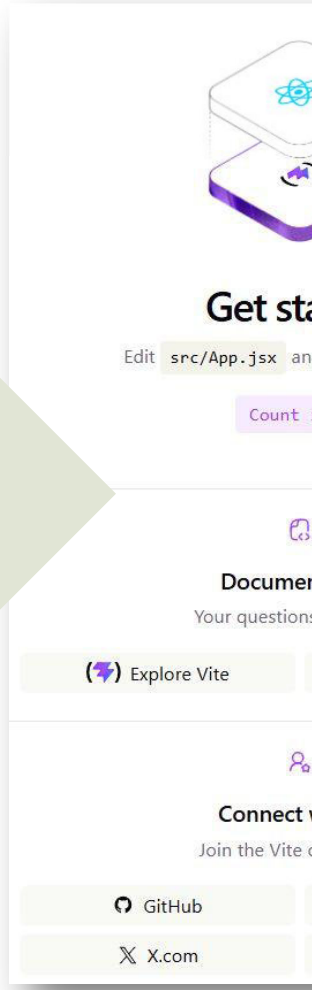


Install Dependency

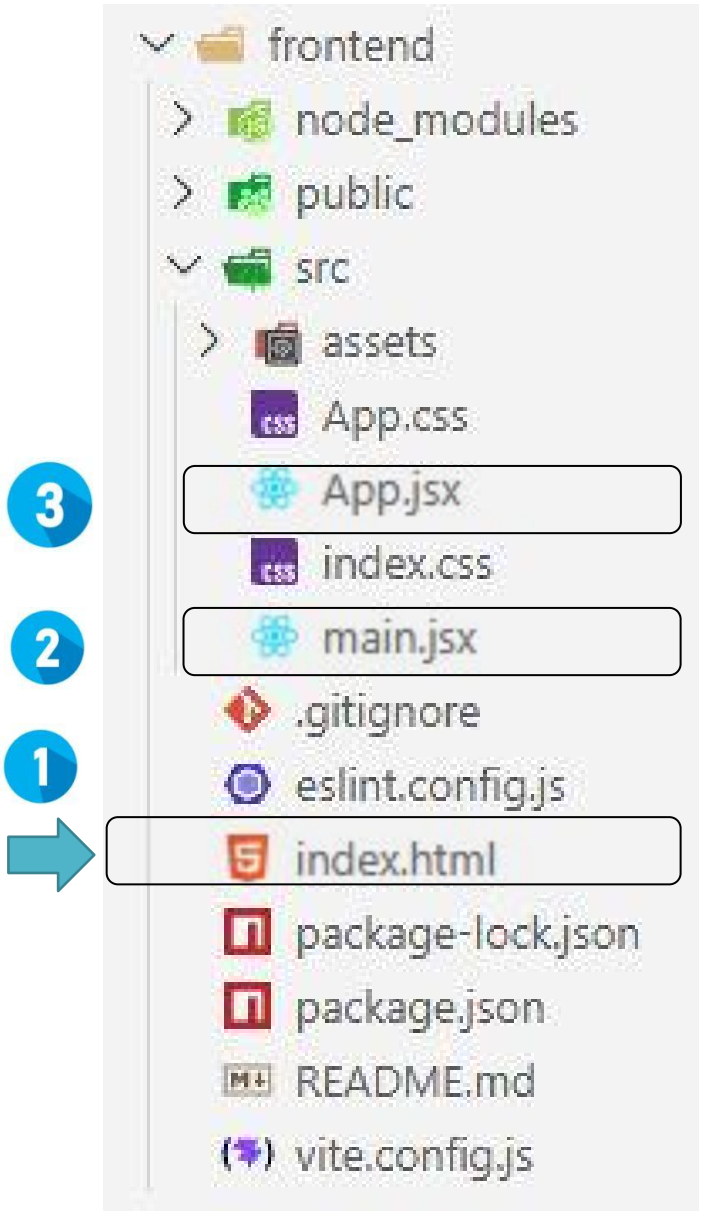
cd vite-project
npm install -y

Run Application

npm run dev



Note : Vite is a frontend tool that is used for building fast and optimized web applica



1 index.html

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

5

2

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

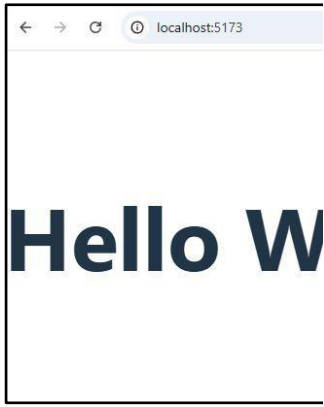
3

App.jsx

```
import './App.css'
function App() {
  return (
    <div className="App">
      <h1>Hello World!</h1>
    </div>
  )
}
```

Return HTML

4



createRoot Function

React uses a **container** to render HTML in a web page

containte
r

```
<div id="root"></div>
```

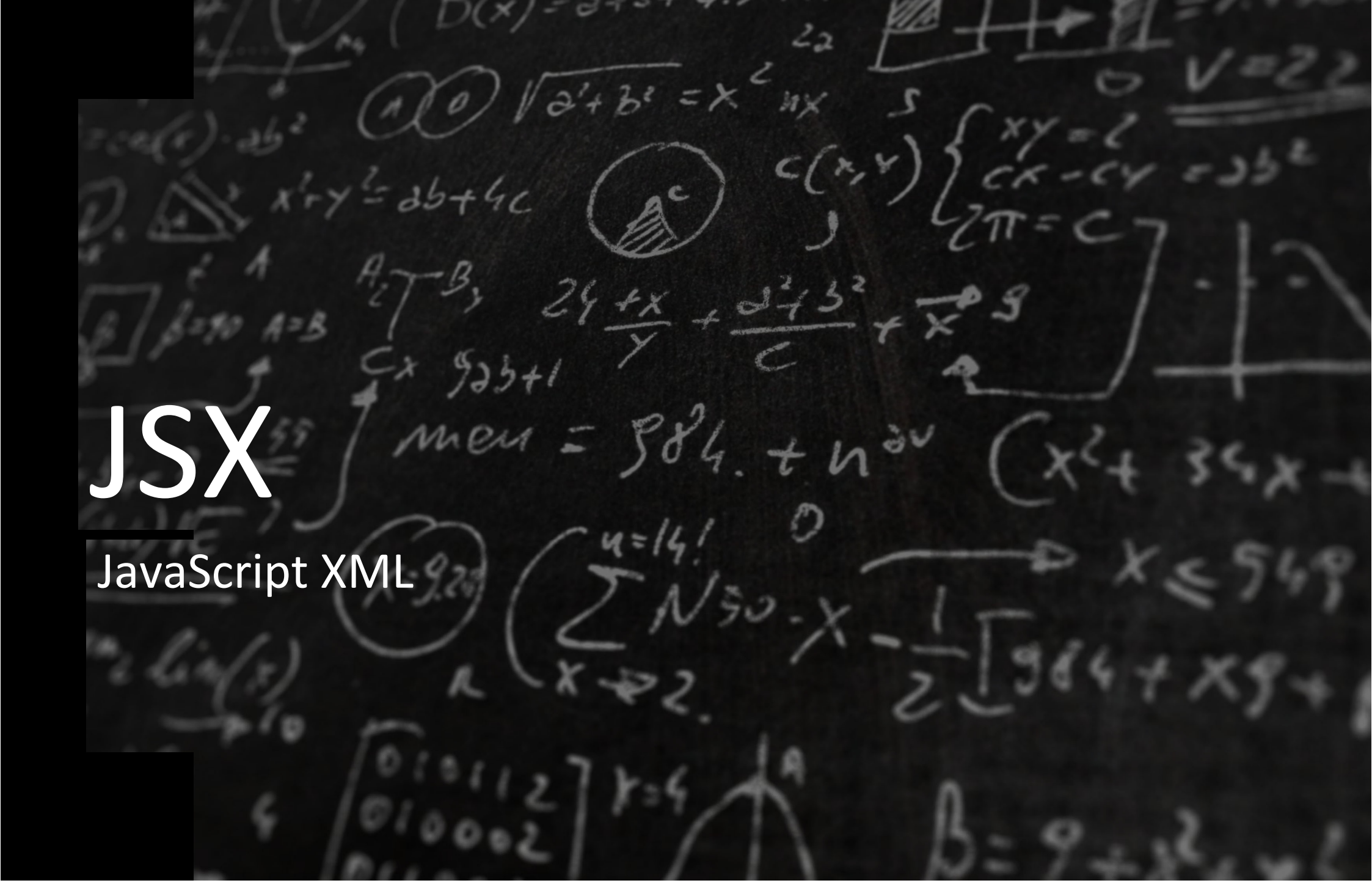
createRoot is built-in function that is used to create a root node for a React app

```
createRoot(containter).render(HTML
)
```

main.jsx

```
import { createRoot } from 'react-dom/client'

createRoot(document.getElementById('root')).render(
  <h1>Hello React!</h1>
)
```



JSX

JavaScript XML

JSX - JavaScript XML

JSX allows us to write HTML directly in React.

JSX write HTML elements in JavaScript and place them in the DOM without any createElement() and/or appendChild() methods

Without JSX:

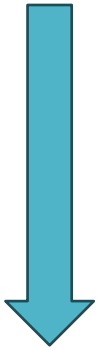
```
import React from 'react';

const myElement = React.createElement('h1', {}, 'I do not use JSX!');
```

With JSX:

```
const myElement = <h1>I Love JSX!</h1>;
```





```
createRoot(document.getElementById('root')).render(myElement);
```

JSX in React Components

React components are like JavaScript functions, and return HTML

JSX works perfect inside React components

```
import { createRoot } from 'react-dom/client'

function SayHi() {
  const name = 'John';
  const lastName = 'Doe';
  return <h1>Hello, {name} {lastName}</h1>;
}

createRoot(document.getElementById('root')).render(
  <SayHi />
)
```

Component – functio



function name **must** start with a **capi**
letter

When defining a component using a JavaScript function, the function name **must** start with a **capital let**

Comments - {/* */}

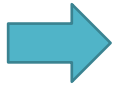
```
{/* add Comment */}
```

Expressions - write expressions inside curly braces { }

```
const myElement = <h1>React is {5 + 5} times better with JSX</h1>;
```

Close empty elements with />

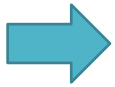
```
const myElement = <input type="text" />;
```



```
<br/>
<hr/>
<img  />
<input />
```

Change some attribute name

```
const myElement = <h1 className="myclass">Hello World</h1>;
```



class => className

Inserting a Large Block of HTML or multiple lines, put the HTML inside parenthe

Single line HTML

```
return <h1>Hello, JSX!</h1>;
```

Multiple line HTML

```
const myElement = (
  <ul>
    <li>Apples</li>
    <li>Bananas</li>
    <li>Cherries</li>
  </ul>
);
```

One Top Level Element - wrapped HTML in ONE top level element

```
const myElement = (
  <div>
    <p>I am a paragraph.</p>
    <p>I am a paragraph too.</p>
  </div>
);
```

```
const myElement = (
  <>
    <p>I am a paragraph.</p>
    <p>I am a paragraph too.</p>
  </>
);
```

<> </> - "fragment"prevent unnecessarily adding extra nodes

Expressions

insert any valid JavaScript expression inside JSX by wrapping it in curly braces { }

expression

```
function Car() {
  return <p>My car has {218 * 1.36} horsepower</p>;
}
```

variable

```
function Car() {
  const hp = 218 * 1.36;
  return <p>My car has {hp} horsepower</p>;
}
```

function

```
function kwtohp(kw) { return kw * 1.36; }

function Car() {
  return <p>It has {kwtohp(218)} horsepower</p>;
}
```

object

```
function Car() {
  const myobj = {
    name: "Fiat",
    model: "500",
    color: "white"
  };

  return (
    <>
      <h1>My car is a {myobj.color} {myobj.name}
{myobj.model}</h1>
    </>
  );
}
```

Attribute

using className instead of class attribute

```
function Car() {
  return <h1 className="myclass">Hello World</h1>;
}
```

Event attributes in JSX are written in camelCase

```
function AlertBtn() {
  const myfunc = () => {
    alert('Hello World');
  };
  return <button onClick={myfunc}>Click me</button>;
}
```

Style attribute

```
function Car() {
  const mystyles = {
    color: "red",
    fontSize: "20px",
    backgroundColor: "lightyellow",
  };

  return <h1 style={mystyles}>My car</h1>;
}
```

- The styles are stored in an object.
- Style properties are written in camelCase, e.g. fontSize, instead of font-size.

Boolean

```
function MyBtn() {
  const myfunc = () => {
    alert('Hello World');
  };
  return <button onClick={myfunc} disabled>Click me</button>;
}

createRoot(document.getElementById('root')).render(
  <MyBtn />
)
```

Set to true => Pass true or pass no value for an attribute

```
return <button onClick={myfunc} disabled>Click me</button>;
```

```
return <button onClick={myfunc} disabled={true}>Click me</button>;
```

Set to false = > Pass false

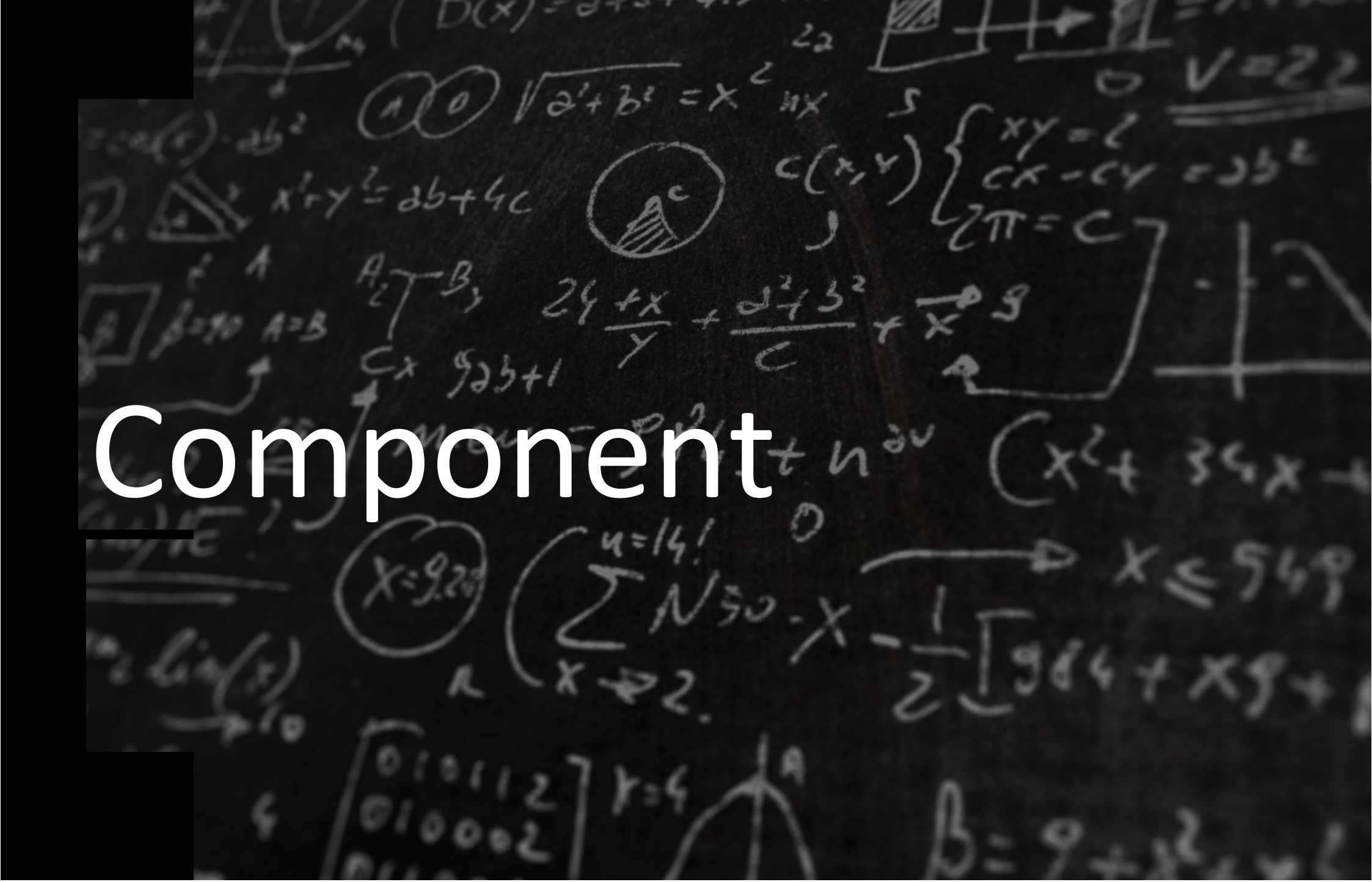
```
return <button onClick={myfunc} disabled={false}>Click me</button>;
```

If Statements

React supports if statements, but not inside JSX

Use ternary expressions instead:

```
function Fruit() {
  const x = 5;
  return (
    <h1>{(x) < 10 ? "Banana" : "Apple"}</h1>
  );
}
```

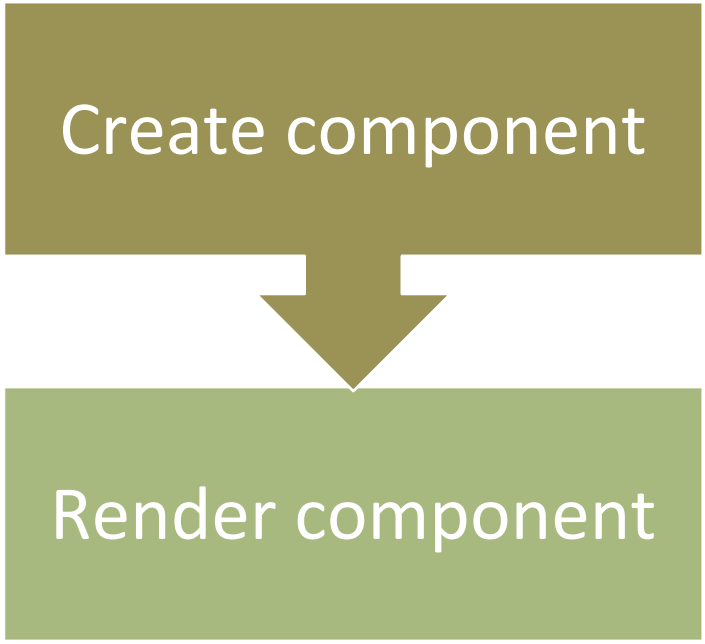


Component

Component

Components are like functions that **return HTML elements**

Component's name MUST start with an **upper case letter**




```
import { createRoot } from 'react-dom/client'

function Hello() {
  return <h1>Hello, world!</h1>;
}


createRoot(document.getElementById('root')).r
  <Hello />
)
```

Props

Props is properties or arguments passed into a component

Strings can be sent inside quotes " "

numbers, variables, and objects sent inside curly brackets **{ }**.

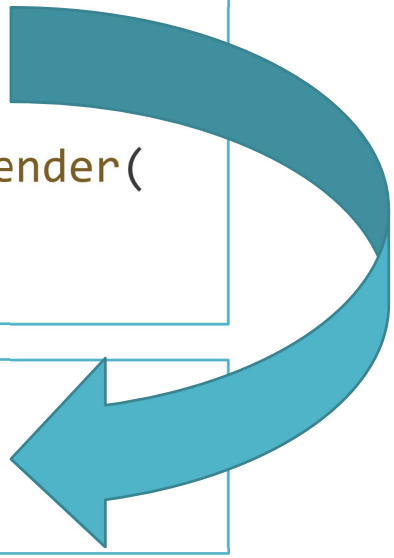
```
function Hello(props) {
  return <h1>Hello, {props.first} {props.last} ({props.age})</h1>;
}
createRoot(document.getElementById('root')).render(
  <Hello first="John" last="Doe" age={35}/>
)
```



Note: React Props are
read-only!

Destructuring props

```
function Hello(props) {
  const { first, age } = props;
  return <h1>Hello, {first} ({age})</h1>;
}
createRoot(document.getElementById('root')).render(
  <Hello first="John" last="Doe" age={35} />
)
```



```
function Hello({first,age}) {
  return <h1>Hello, {first} ({age})</h1>;
}
```

```
function Hello({first}) {
  return <h1>Hello, {first}</h1>;
}
```

Set default value

```
function Hello({ first, age , type="student"}) {
  return <h1>Hello, {first} ({age}) {type}</h1>;
}
```

Object props

```
import { createRoot } from 'react-dom/client'

function Hello(props) {
  return (
    <h1>Hello, {props.person.first} {props.person.last} ({props.person.age})<
  );
}

const person = {
  first: 'John',
  last: 'Doe',
  age: 35
};

createRoot(document.getElementById('root')).render(
  <Hello person={person} />
)
```

Hello, John Doe (35

Array

props

```
import { createRoot } from 'react-dom/client'

function MyFruits(props) {
  return <h1>I love {props.fruits[0]} {props.fruits[1]} {props.fruits[2]}</h1
}

const fruits = ['Apple', 'Banana', 'Cherry'];
createRoot(document.getElementById('root')).render(
  <MyFruits fruits={fruits} />
)
```

I loveApple Banana C

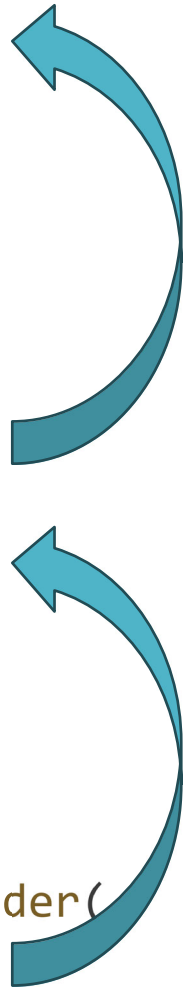
```
function MyFruits(props) {
  return (
  <ul>I love
    {props.fruits.map((fruit) => (<li key={fruit}>{fruit}</li>))}
  </ul>);
}
```

Component in Component

```
function Fruit(props) {
  return <li>{props.fruit}</li>;
}

function ShowFruits(props) {
  return (<div>
    <h1>My favourite fruits</h1>
    <ul>
      <Fruit fruit={props.fruits[0]} />
      <Fruit fruit={props.fruits[1]} />
      <Fruit fruit={props.fruits[2]} />
    </ul>
  </div>);
}

const fruits = ['Apple', 'Banana', 'Cherry'];
createRoot(document.getElementById('root')).render(
  <ShowFruits fruits={fruits} />
)
```



My favourit

- Apple
- Banana
- Cherry

Props Children

props.children property used to access the content that send from parent component

```
import { createRoot } from 'react-dom/client'

function Child(props) {
  return (
    <div style={{border: '2px solid blue', margin: '10px', padding: '10px'}}>
      <h2>My Child</h2>
      {props.children}
    </div>
  );
}
function Parent() {
  return (
    <div>
      <h1>My Children</h1>
      <Child><p>this is the first child.</p></Child>
      <Child><p>this is the second child.</p></Child>
    </div>
  );
}
createRoot(document.getElementById('root')).render(<Parent />)
```



List

List can be render with some type of loop

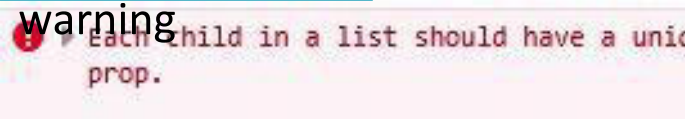
map() array method is generally the preferred method.

```
function ShowFruits() {
  const fruits = ["Apple", "Banana", "Cherry"];
  return (<>
    <h1>My favourite fruits</h1>
    <ul>
      {fruits.map((fruit) => <li>{fruit}</li>)}
    </ul>
  </>);
}
```

My favourite fruits

- Apple
- Banana
- Cherry

no "key“ warning



Keys

key should be a **unique ID** assigned to each item to keep track of elements
If an item is updated or removed, only that item will be re-rendered instead of the entire list.
you can use the **array index** as a key.

```
    <ul>
      {fruits.map((fruit, index) => <li key={index}>{fruit}</li>)}
    </ul>
```

List object

```
function MyFruits() {
  const fruits = [
    { id: 1001, name: 'Apple' },
    { id: 1002, name: 'Banana' },
    { id: 1003, name: 'Cherry' }
  ];
  return (<>
    <h1>My sdsd favourite fruits</h1>
    <ul>
      {fruits.map(fruit => <li key={fruit.id}>{fruit.name}</li>)}
    </ul>
  </>);
}
```

Condition

use if operator to decide which component to render

```
import { createRoot } from 'react-dom/client'

function Welcome() { return <h3>Welcome</h3>;
}
function Error() { return <h3>User undefined</h3>;
}

function CheckLogin({ isLogin }) {
  if (isLogin) {
    return <Welcome />
  }
  return <Error />
}
```

Ternary Operator

```
function CheckLogin({ isLogin }) {
    return isLogin ? <Welcome /> : <E
}
```

```
isLogin={true}
Welcome
```

```
isLogin={false
User undefin
```

Logical && Operator

using the **&&** operator to check the empty value

the component will only be rendered if the property is not empty or define as true:

```
function Greeting({ userName }) {
  return <h3>Member : {userName}</h3>;
}

function CheckLogin({ userName }) {
  return (<>
    <h1>Welcome to our page </h1>
    {userName && <Greeting userName={userName} />}
  </>);
}

createRoot(document.getElementById('root')).render(
  <CheckLogin userName="John" />
)
```

Welcome to our page

Member : John

```
<CheckLogin />
<CheckLogin userName=
<CheckLogin userName=
```

Welcome to ou

```
<CheckLogin userName=
```

Welcome to ou

Member :

```
<CheckLogin userName=
```

Welcome to ou

Member : 1

```
<CheckLogin userName=
```

Welcome to ou

0

Render

Event

React events are written in camelCase syntax onClick

React event handlers are written inside { }

JSX

```jsx
<button onClick={hi}>Take the Shot!</button>
```

HTML

```html
<button onclick="hi()">Take the Shot!</button>
```

```jsx
function greeting() {
  alert(`Hello , nice to see you!`);
}

function Welcome() {
  return (<>
    <h1>Welcome to our page </h1>
    <button onClick={greeting}>click</button>
  </>);
```

Passing Arguments

To pass an argument to an event handler, use an arrow function

```
function Welcome() {
  const greeting = (first, last) => {
    alert(`Hello ${first} ${last}`);
  }

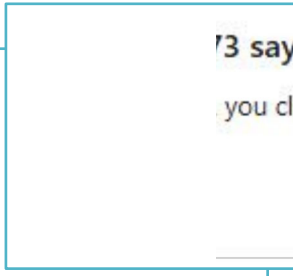
  return (<>
    <h1>Welcome to our page </h1>
    <button onClick={()=>greeting("John","Doe")}>click</button>
  </>);
```



Event Object

Sent React event to manipulate to the function

```
function Welcome() {
  const greeting = (first, last, e) => {
    alert(`Hello ${first} ${last}, you ${e.type} the button!`);
  }
  return (<>
    <h1>Welcome to our page </h1>
    <button onClick={(event)=>greeting("John","Doe",event)}>click</button>
  </>);
}
```



External component

External components can be define in separate files and imported and used wit other components.

Header.js

```
const Header = ({ shopName }) => {
    const css = {
        backgroundColor: "lightblue",
        padding: "10px",
        textAlign: "center"
    };
    return (<div style={css}>
        <h1>{shopName}</h1>
        <p>Welcome to our webPage</p></div>
    );
}
export default Header;
```

main.jsx

```
import { createRoot } from 'react-dom/cl
import Header from './Header';

const shopName = "CPE bakery";

createRoot(document.getElementById('root
  <Header shopName={shopName} />
)
```



Body.js

```jsx
const Body = ({products}) => {
    const css = {
        border: "1px solid lightblue",
        padding: "10px",
        height: "200px",
    };
    return (<div style={css}>
        <h5>Our Products</h5>
        <ul>
            {products.map((item, index) => <
        </ul>
    </div>
    );
}

export default Body;
```



main.jsx

```jsx
import { createRoot } from 'react-dom/client'
import Header from './Header';
import Body from './Body';

const shopName = "CPE bakery";
const products = ["cake", "bread", "milk"];

createRoot(document.getElementById('root')).r
  <>
    <Header shopName={shopName} />
    <Body products={products} />
  </>
```



CSS

CSS stylesheets

Import CSS styling file (.css) in your application.

style.css

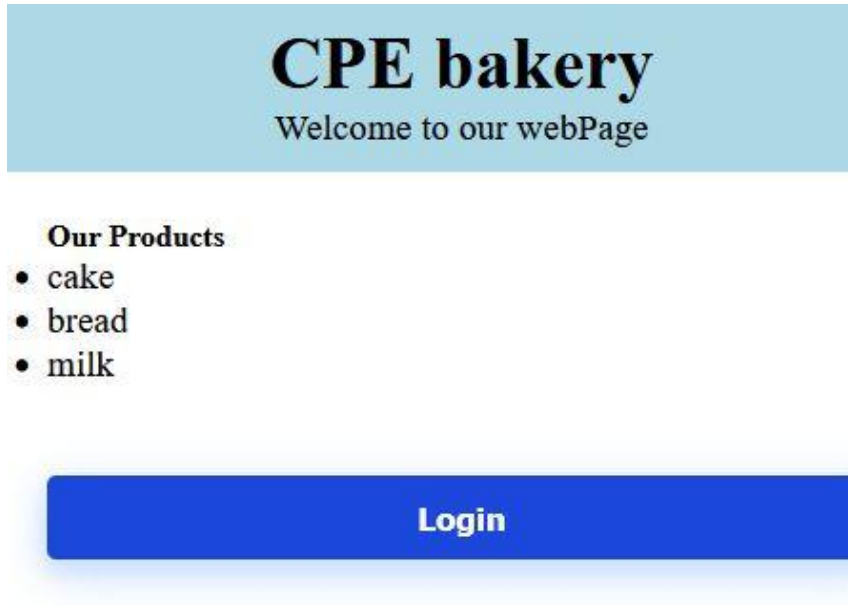
```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
button {
  border: none;
  width: 100%;



}
```

main.css

```
import './style.css'
```



Inline Styling

To style with the inline style attribute, the value must be a **JavaScript object**

Properties name with camelCase

```
const Header = ({ shopName }) => {
    const css = {
        backgroundColor: "lightblue",
        padding: "10px",
        textAlign: "center",
        color: "darkblue"
    };
    return (<div style={css}>
        <h1 style={{ color: "blue" }}>{shopName}</h1>
        <p>Welcome to our webPage</p></div>
    );
}
export default Header;
```

background-color -> backgroundColor

text-align -> textAlign



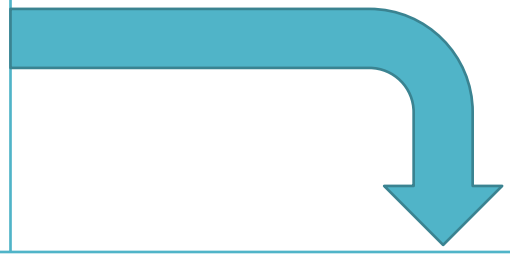
CSS Module

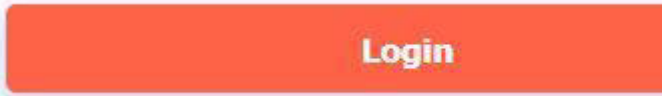
CSS Modules is scoped locally to a specific component

CSS module file have the .module.css extension

```css
.mybutton {
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  background-color: tomato;
}
```

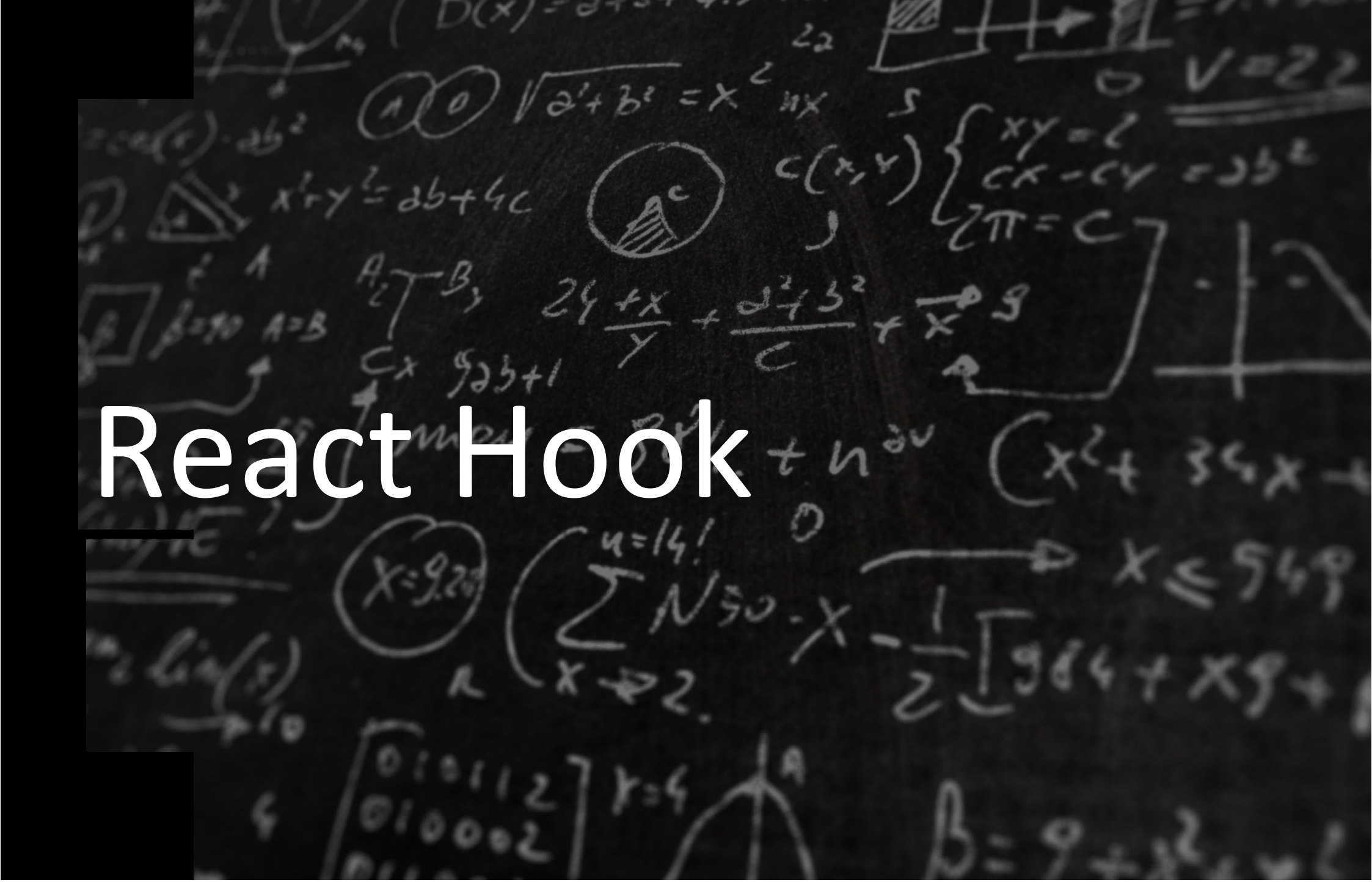
Button.module.css






```jsx
import styles from './Button.module.css';

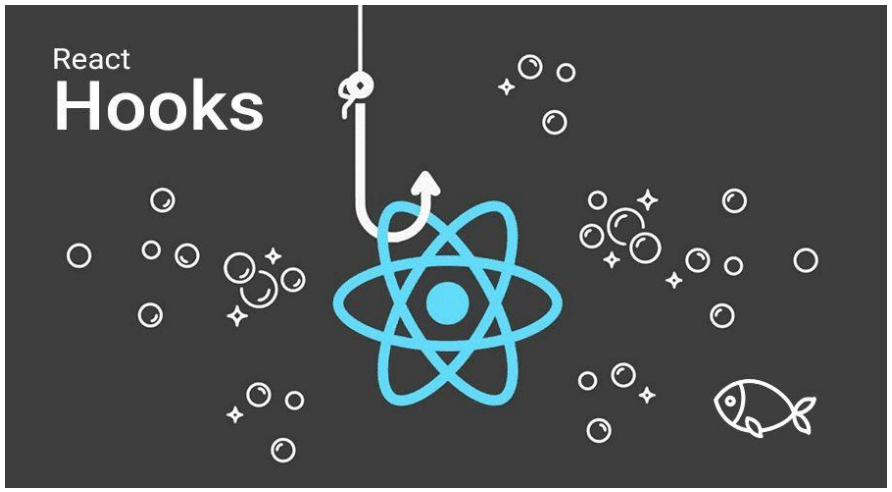
const Btn = () => {
    return <button className={styles.mybutton}>Login</b
}
```

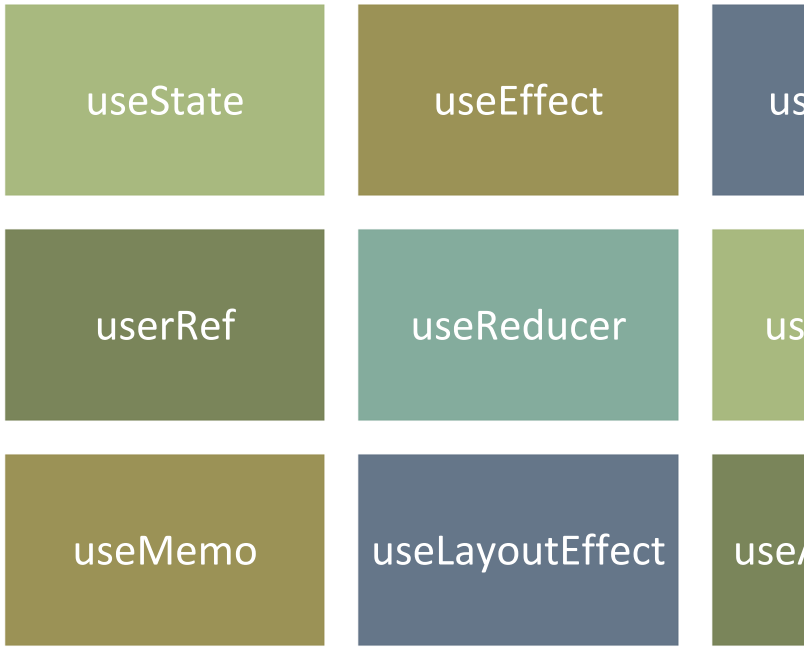


React Hook

React Hooks

React Hooks are functions that enable you to "hook into" React state and lifecyc
features directly from function components





Hooks can only be called inside React function components.
Hooks can only be called at the top level of a component.
Hooks cannot be conditional

useState Hook – data binding

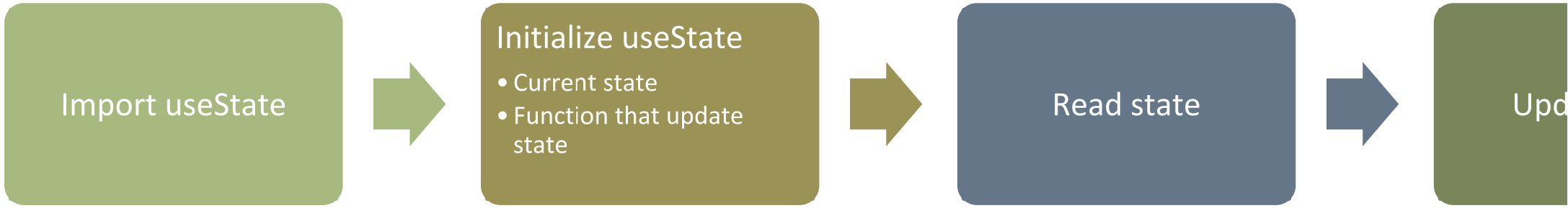
Data binding is a technique that binds data sources from the provider and cons
together and synchronizes them.

React handles data binding by using a unidirectional data flow,
where state and props manage component data, and controlled components en
real-time synchronization between the UI and state

One-way Data Binding	Two way Data Binding
• Props • Parent component pass data to a child component • child component can read these props but cannot modify	• State • Parent component pass data to a child component • Child linking the new value to a state variable and updating that state variable with event handle

useState Hook

The React useState Hook used to track state in a function component




```
import { useState } from 'react'; // import

const [state, setStateFunc] = useState("initState"); //initialize

{state}  //put state in component

setStateFunc("newState")  //update state
```

```jsx
import { createRoot } from 'react-dom/client';
import { useState } from 'react';

function FavoriteColor() {
  const [color, setColor] = useState("red");
  return (
    <>
      <h1 style={{color:color}}>My favorite color is {color}!</h1>
      <button type="button" onClick={() => setColor("red")}>Red</button>
      <button type="button" onClick={() => setColor("blue")}>Blue</button>
      <button type="button" onClick={() => setColor("green")}>green</button>
    </>
  )
}
```



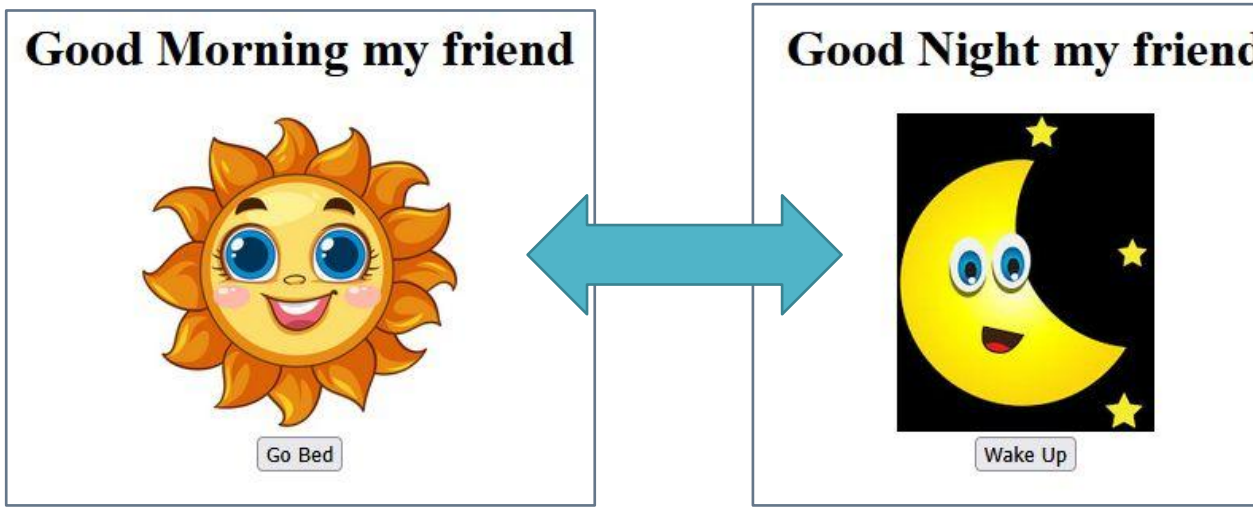




Add color input → `<input type="color" onChange={(e) => setColor(e.target.value)} />`

```jsx
import { useState } from 'react';
import sun from './assets/sun.png'
import moon from './assets/moon.png'

function DayNight() {
    const [isDay, setIsDay] = useState(true);
    return (
        <div style={{ textAlign: "center" }}>
            <h1> {isDay ? "Good Morning" : "Good Night"} my friend</h1>
            <div><img src={isDay ? sun : moon} height="200px" alt="" /></div>
            <button type="button"
                onClick={() => setIsDay(!isDay)}>{isDay ? "Go Bed" : "Wake Up"}</bu
        </div>
    );
}
export default DayNight;
```



Updating Arrays in

```
import { useState } from 'react';
const Body = () => {
    const [products, setProducts] = useState(["cake", "bread", "milk"]);

    const deleteItem = (index) => {
        const result = products.filter((item, i) => i !== index);
        setProducts(result);
    }

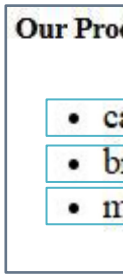
    return (<div style={css}>
        <h5>Our Products</h5>
        <ul>
            {products.map((item, index) => (<li key={index}>{item}
                <button onClick={() => deleteItem(index)}> del</button></li>)
            )}
        </ul>
    </div>
    );
}

export default Body;
```



```
        <ul>
            {products.map((item, index) => (<li key={index}>{item}
                <button onClick={() => deleteItem(index)}> del</button></li>)
            )}
        </ul>
```

Body



Pass function to container

```
<ul>
     { products.map((item, index) => (
            <Product item={item} index={index} deleteItem={deleteItem} />))
     }
</ul>
```

Product.js

```
const Product=({item, index, deleteItem})=>{
    return <li>{item} <button onClick={()=>deleteItem(index)}> del</button></li>
}
export default Product;
```

Create Form

ProductForm.js

```
import { useState } from 'react';


function ProductForm() {

  function handleSubmit(e) {
    e.preventDefault();
    alert("hello, You clicked submit. " );
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Enter new product:
        <input type="text" />
      </label>
      <input type="submit" />
    </form>
  )
```



Get value from input

```
function ProductForm() {
  const [name, setName] = useState("");

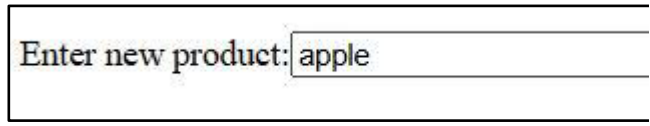
  function handleSubmit(e) {
    e.preventDefault();
    alert("Add product " + name);
  }

  function handleChange(e) {
    setName(e.target.value);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Enter new product:
        <input type="text" value={name} onChange={handleChange}/>
      </label>
      <input type="submit" value="Submit"
                          disabled={name.trim()===""}/>
```



Prevente to load new page



1

2

3

4

5



Add item to array

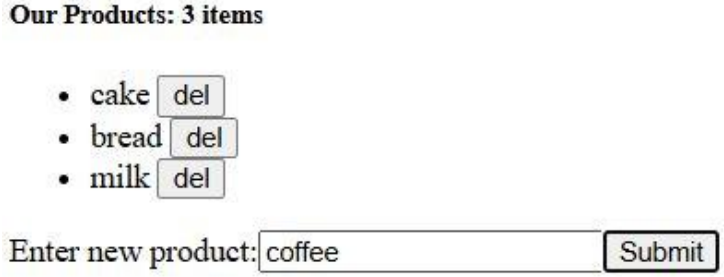
Body

```
const [products, setProducts] = useState(["cake", "bread", "milk"]);

const addItem = (item) => {
    const result = [...products, item];
    setProducts(result);
}

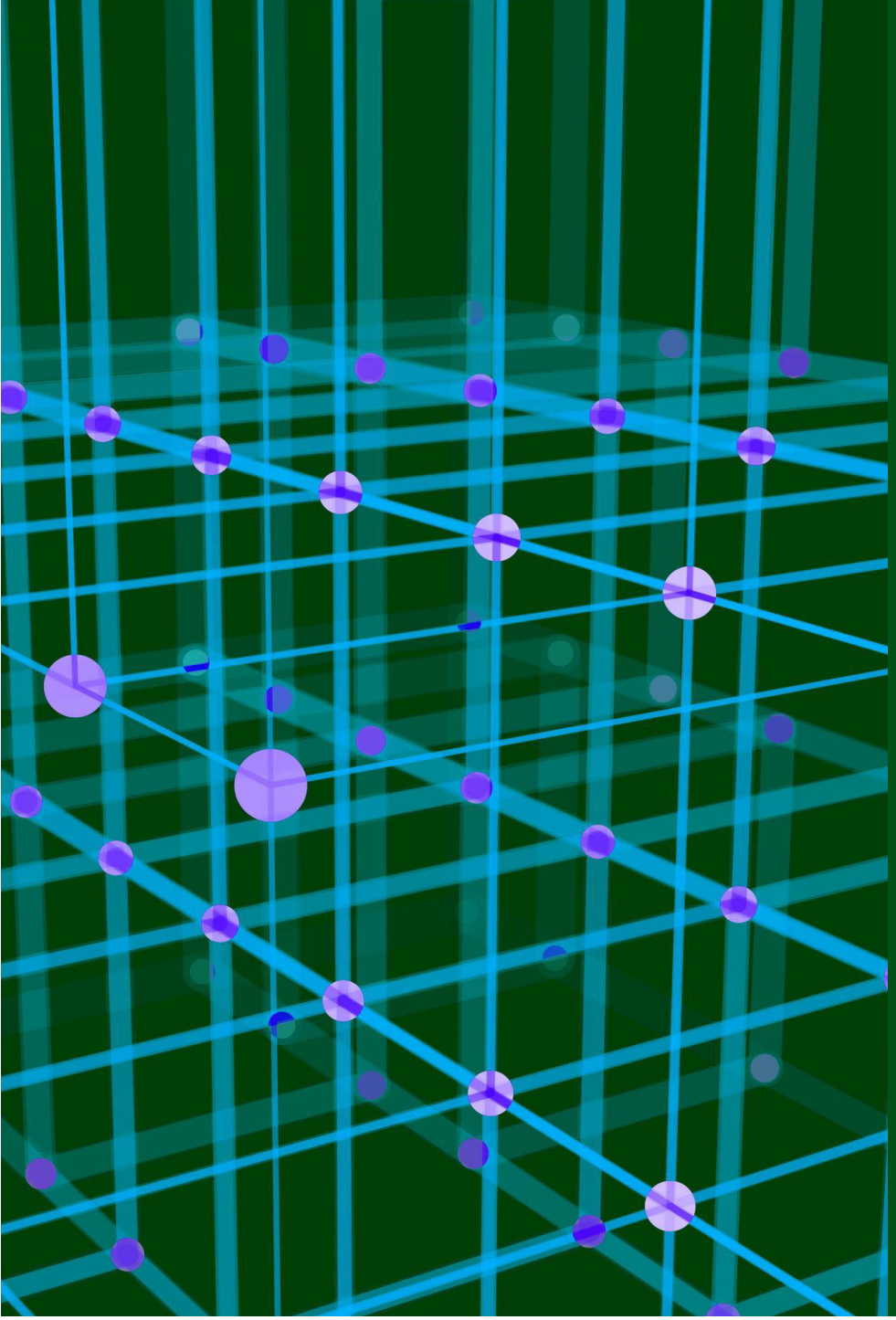
//return
<ProductForm addItem={addItem}/>
```






```
function ProductForm({addItem}) {
  const [name, setName] = useState("

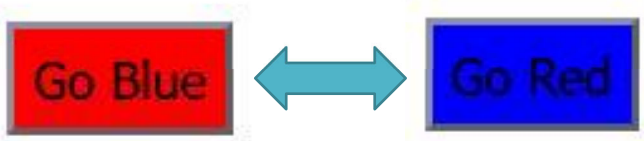
  function handleSubmit(e) {
    e.preventDefault();
    addItem(name);
    setName("");
  }
```



Todo

Get start with React

1. เปลี่ยนสี button และข้อความ เมื่อกดปุ่ม



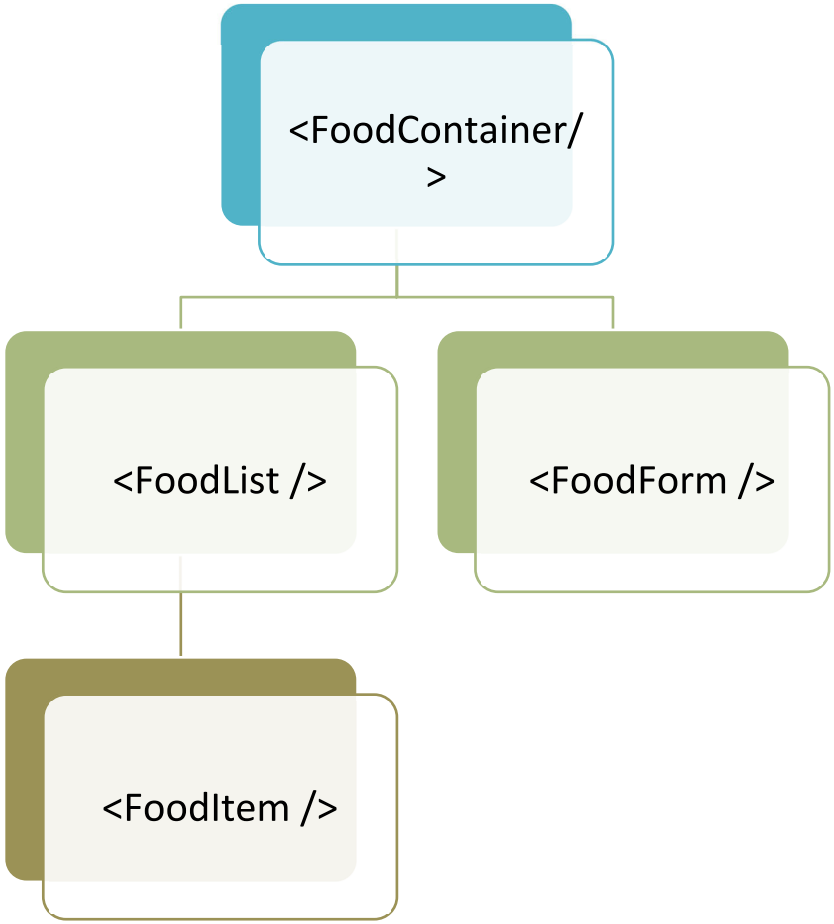
2. แสดงข้อความและสีตามที่กำหนด (onChange)



3. แสดงตัวเลข เพิ่ม / ลด ตามการกดปุ่ม

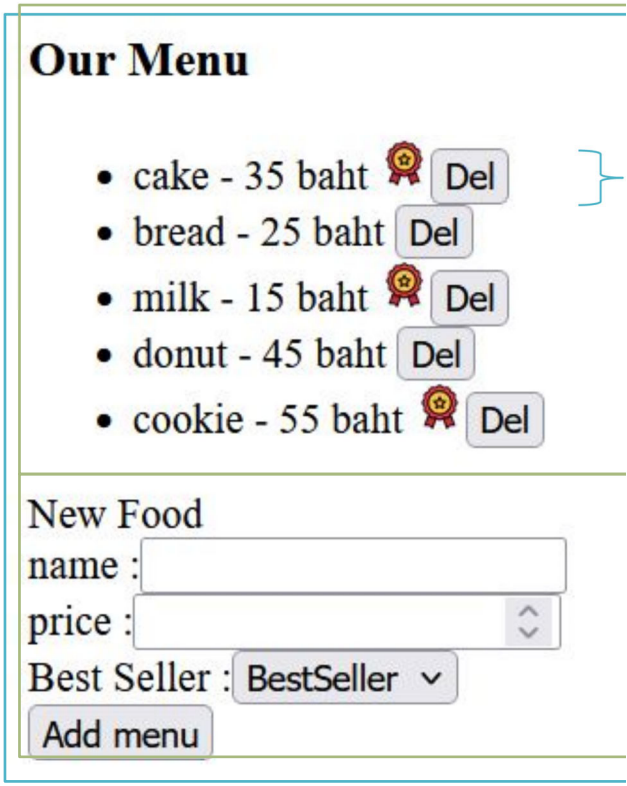


1. Menu management



FoodList

FoodForm



Updating Arrays Object in

```
food = [
    { name: "cake", price: 35, isBestSeller: true },
    { name: "bread", price: 25, isBestSeller: false
    { name: "milk", price: 15, isBestSeller: true },
    { name: "donut", price: 45, isBestSeller: false
    { name: "cookie", price: 55, isBestSeller: true
];
```

FoodContainer

```
Import


const FoodContainer = () => {
    const [food, setFood] = useState(

    );

    const deleteItem = (index) => {    }

    const addItem = (item) => {    }

    return (<div>
        <h3>Our Menu</h3>
        <FoodList                          />
        <FoodForm                          />
    </div>
    );
}
export default FoodContainer;
```

FoodList

```
Import

const FoodList = (                    ) => {
    return (
        <div>
            <ul>
                {food.map((           ) =>
                    <FoodItem                          />
                )}
            </ul>
        </div>
    );
}
export default FoodList;
```

Instead of trying to render the entire object, submit the **specific properties**

```
item={item}
```



```
name={item.name}
price={item.price}
```

FoodItem

```
Import

const FoodItem = (                    ) => {
    return (<li key={     }>




        <button onClick={                 }>Del</button>
    </li>);
}

export default FoodItem;
```

FoodForm

```
Import

const FoodForm = (           ) => {
    const [inputs, setInputs] = useState({name:"", price:"",isBestSeller: 'true'

    function handleChange(e) {
        const name = e.target.name;
        const value = e.target.value;
        setInputs(values => ({ ...values, [name]: value }));
    }

    function handleSubmit(e) {
        e.preventDefault();
        const newFood = {



        };
        setInputs({name:"",price:"",isBestSeller: 'true'});

    }
```

FoodForm

```
    return (<form onSubmit={          }>

        <input type="text" name="name" value={inputs.name} onChange={handleChang


        <input type="number" name="price" value={inputs.price} onChange={handleC


        <select name="isBestSeller" onChange={handleChange}>
            <option value="true">BestSeller</option>
            <option value="false">Normal</option>
        </select>


        <button>Add menu</button>

    </form>);
}
export default FoodForm;
```

2. User / admin mode

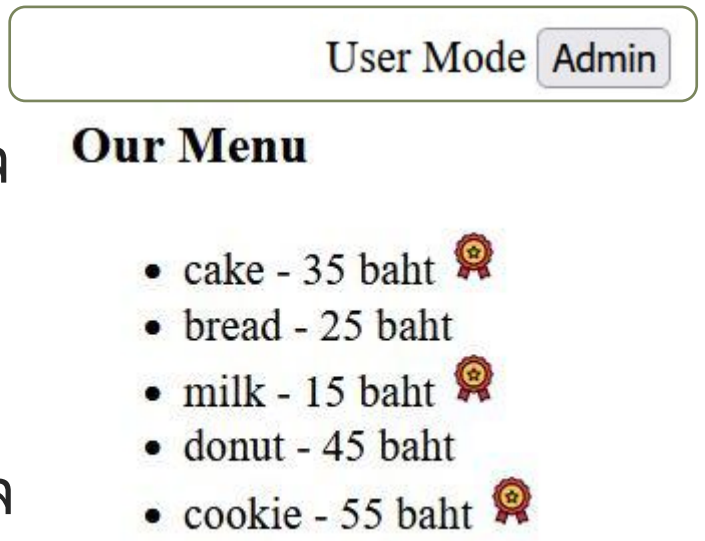
USER MODE

ซ่อนปุ่มลบ และฟอร์มเพิ่มข้อมูล

ADMIN MODE

แสดงปุ่มลบ และฟอร์มเพิ่มข้อมูล

ใช้ localStorage บันทึก Mode ปัจจุบันไว้





3. CSS

ปรับแต่งโดยใช้ CSS